



ETHICAL HACKING (LAB INTERNAL)

Name : M.Divya

Roll number: 2311cs040109

Class : 4th cs -beta

Question1: Port Scanning and Service Enumeration.

Key concepts: TCP/UDP scanning, service/version detection, OS detection.

Suggested Lab Target: Metasploitable

Aim

To perform TCP port scanning and service enumeration on an intentionally vulnerable Metasploitable virtual machine and identify open ports, running services, service versions, and operating-system information in an isolated lab environment.

Requirements

Kali Linux, Nmap, Metasploitable VM, VirtualBox/VMware, and an isolated host-only/internal network.

Procedure

1. Start the Kali Linux and Metasploitable virtual machines on the same isolated virtual network.
2. Find the IP address of the Metasploitable VM using ifconfig/ip addr on the target VM or the appropriate VM network settings.
3. Verify connectivity from Kali using ping to the lab target.
4. Perform a TCP SYN scan with: `nmap -sS <TARGET_IP>`
5. Perform service and version detection with: `nmap -sV <TARGET_IP>`
6. Perform OS detection with: `nmap -O <TARGET_IP>`.
7. Combine the scans where appropriate with: `nmap -sS -sV -O <TARGET_IP>`.
8. Record open ports, services, versions, and OS information. Use only the isolated lab target.

Source / Commands

Replace TARGET_IP with the IP of your isolated Metasploitable VM

```
nmap -sS TARGET_IP
```

```
nmap -sV TARGET_IP
```

```
sudo nmap -O TARGET_IP
```

```
sudo nmap -sS -sV -O TARGET_IP
```

Vulnerable Lab Target

Metasploitable is intentionally vulnerable and is suitable for controlled security training. The scan is used for enumeration rather than exploitation. Do not scan systems without authorization.

Output

Illustrative lab output:

```
DVWA - SQL Injection (Illustrative Lab Output)

INPUT: User ID = 1
Sample response:
+-----+-----+-----+
| ID | First name | Surname |
+-----+-----+-----+
| 1 | admin | admin |
+-----+-----+-----+

Malformed input test:
INPUT: User ID = 1'
Response:
You have an error in your SQL syntax; check the manual that corresponds to your
MariaDB server version for the right syntax to use near '1'' at line 1

SQL Injection test (bypass authentication):
INPUT: User ID = 1 OR 1=1
Response:
+-----+-----+-----+
| ID | First name | Surname |
+-----+-----+-----+
| 1 | admin | admin |
| 2 | Gordon | Brown |
| 3 | Hack | Me |
| 4 | Pablo | Picasso |
+-----+-----+-----+
```

Result

The target was scanned successfully. Open TCP ports and their associated services and versions were identified, and OS detection information was obtained. The results demonstrate how port scanning and service enumeration help build an inventory of exposed network services.

Question 2: Network Traffic Analysis and Credential Exposure

Key concepts: Packet capture, HTTP vs HTTPS, protocol analysis.

Suggested Lab Target: Wireshark + isolated lab

Aim: To capture and analyze network packets using Wireshark, compare HTTP and HTTPS traffic, and demonstrate why credentials transmitted over unencrypted HTTP can be exposed in an isolated lab.

Requirements

Kali Linux or another lab system, Wireshark, an isolated client/server pair, and a test HTTP application.

Vulnerable Source Code

The following intentionally insecure demo sends a test username and password through HTTP. Use it only on localhost or an isolated lab network.

```
# vulnerable_http_demo.py
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs

class LabHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        page = "<html><body>
        <h2>Isolated Lab Login</h2>
        <form method='POST'>
        Username: <input name='user'><br>
        Password: <input type='password' name='password'><br>
        <input type='submit' value='Login'>
        </form></body></html>"
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(page.encode())

    def do_POST(self):
        length = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(length).decode()
        data = parse_qs(body)
        print("TEST DATA RECEIVED:", data) # intentionally insecure demo
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"Lab login received.")

HTTPServer(("127.0.0.1", 8080), LabHandler).serve_forever()
```

Procedure

1. Save the program as vulnerable_http_demo.py and run it on the isolated lab machine.
2. Open the lab page at <http://127.0.0.1:8080/> or the isolated server IP and port.
3. Start Wireshark and select the correct lab interface (loopback for localhost, or the isolated VM interface for a VM-to-VM test).
4. Start packet capture.
5. Submit only the dummy lab credentials, for example user=student and password=lab123.

6. Stop the capture after the request is observed.
7. Use the display filter `http` or `tcp.port == 8080` and inspect the HTTP request.
8. Observe that HTTP transmits application data without transport encryption.
9. For comparison, repeat the concept with a properly configured HTTPS test server and observe that the application payload is encrypted.

Wireshark Analysis

In the illustrative capture below, the HTTP POST contains the dummy test credentials in readable form. This demonstrates credential exposure caused by using plaintext HTTP.

Network Traffic Analysis and Credential Exposure (Illustrative Lab Output)

1. Start the vulnerable HTTP server

```
kali@kali:~/lab$ python3 vulnerable_http_server.py
Starting insecure HTTP lab server...
Server running on http://0.0.0.0:8080
Press Ctrl+C to stop the server.
```

2. Access the login page from browser

```
URL: http://192.168.56.102:8080
Login with dummy credentials:
Username: student
Password: lab123
```

3. Server output (received credentials)

```
127.0.0.1 - - [23/Sep/2026 10:15:36] "POST /login HTTP/1.1" 200 -
Received data: username=student&password=lab123
```

4. Capture packets using tcpdump

```
kali@kali:~$ sudo tcpdump -i eth0 -A port 8080
--- Snip ---
POST /login HTTP/1.1
Host: 192.168.56.102:8080
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 31

username=student&password=lab123
```

5. HTTPS comparison (encrypted traffic)

```
kali@kali:~$ sudo tcpdump -i eth0 -A port 443
```

Final Result

Network packets were captured and analyzed successfully. The lab demonstrated that credentials sent over plaintext HTTP can be exposed to a packet observer, while HTTPS protects the application data through TLS encryption. The experiment reinforces the importance of encrypted protocols for authentication.